

Automated Forecasting Systems

Auto-ARIMA | Auto Prophet | AutoML for Time Series | AutoML + Feature Hashing

1. Time Series & Automated Forecasting — Overview

A time series is a sequence of data points collected at equally spaced time intervals, where order matters and past values influence future ones. Automated forecasting replaces manual model selection, parameter tuning, and stationarity testing with algorithmic pipelines.

Key advantage of automation: one reproducible pipeline can handle hundreds of series simultaneously, reducing expert time from days to minutes.

1.1 Time Series Components

- Trend: Long-term increase or decrease (e.g., rising annual sales).
- Seasonality: Fixed-period repeating cycles — daily, weekly, yearly (e.g., holiday spikes).
- Cyclicity: Irregular long-period fluctuations tied to economic cycles.
- Noise: Random, unexplained residual variation.

Decomposition: $Y(t) = \text{Trend} + \text{Seasonality} + \text{Cyclicity} + \text{Noise}$

2. Auto-ARIMA

2.1 ARIMA — Classical Foundation

ARIMA (AutoRegressive Integrated Moving Average) is the standard statistical model for univariate time series.

- AR (p): Uses past p observations — captures autocorrelation.
- I (d): Differencing applied d times to make series stationary.
- MA (q): Uses past q forecast errors to smooth predictions.

ARIMA(p, d, q) — AR order p · Differencing d · MA order q

Manual selection of (p, d, q) requires reading ACF/PACF plots, running ADF/KPSS stationarity tests, and subjective expert judgment — slow and error-prone.

2.2 How Auto-ARIMA Works

Auto-ARIMA (via the pmdarima library in Python) automates the entire model selection process:

- **Run stationarity test (ADF/KPSS) to determine the differencing order d.** Step 1:
- **Define search space for p and q (e.g., 0 to 5).** Step 2:

- **Fit all candidate ARIMA(p, d, q) models.** Step 3:
- **Evaluate each using AIC or BIC — select the model with the lowest score.** Step 4:
- **Optionally test seasonal extensions (SARIMA) if m is specified.** Step 5:
- **Return best model with confidence intervals.** Step 6:

2.3 AIC & BIC — Model Selection Criteria

$$\text{AIC} = 2k - 2 \ln(\hat{L}) \quad | \quad \text{BIC} = k \cdot \ln(n) - 2 \cdot \ln(\hat{L})$$

- k = number of parameters; $\hat{L}$ = maximised likelihood; n = sample size.
- Lower AIC/BIC = better model (penalises unnecessary complexity).
- BIC penalises more heavily on large datasets → prefers simpler models.

2.4 SARIMA — Seasonal Extension

Basic ARIMA cannot model seasonal data. SARIMA adds seasonal components:

$$\text{SARIMA}(p, d, q)(P, D, Q)_m \quad \text{where } m = \text{seasonal period}$$

- P, D, Q : Seasonal AR, differencing, and MA orders.
- m : Period — 12 for monthly, 7 for weekly, 4 for quarterly, 24 for hourly.
- Example: SARIMA(1,1,1)(1,1,1)₁₂ — monthly data with yearly seasonality.

When to use: 1–50 univariate series with classic linear trend patterns. Fastest inference among all automated methods.

3. Auto Prophet

3.1 What is Prophet?

Prophet is an open-source forecasting tool released by Meta (Facebook) in 2017. It is designed for business analysts — not statisticians — and handles business time series with strong seasonality, holidays, and trend changes.

3.2 Prophet's Decomposition Model

$$y(t) = g(t) + s(t) + h(t) + \epsilon(t)$$

- **Long-term, non-periodic changes. Supports piecewise linear or logistic (S-curve) growth.** $g(t)$ — Growth/Trend:
- **Periodic cycles modelled using Fourier series. Handles daily, weekly, and yearly patterns simultaneously.** $s(t)$ — Seasonality:

- **Effect of known irregular events.** Each holiday gets a separate coefficient with a configurable window. $h(t)$ — Holidays:
- **Unexplained residual, assumed to follow a Laplace distribution (Bayesian framework).** $\epsilon(t)$ — Noise:

3.3 Key Features

- **Changepoint Detection:** Automatically identifies trend shifts using a sparsity prior. Configurable via `n_changepoints` (default = 25) and `changepoint_prior_scale`.
- **Multiple Seasonality:** Yearly, weekly, daily — all handled simultaneously with Fourier series.
- **Holiday Effects:** User specifies a holiday DataFrame with optional lower/upper windows.
- **Additive vs Multiplicative Seasonality:** Additive (default) — amplitude is constant. Multiplicative — amplitude grows with trend (use for exponential growth).
- **Uncertainty Quantification:** Built-in confidence intervals using Monte Carlo simulation.

3.4 Automating Prophet — Grid Search + Cross-Validation

Hyperparameter tuning for Prophet is automated using grid search over key parameters:

- `changepoint_prior_scale`: Controls trend flexibility (0.001 to 0.5).
- `seasonality_prior_scale`: Controls amplitude of seasonal terms.
- `seasonality_mode`: 'additive' or 'multiplicative'.

Walk-forward cross-validation is used for evaluation — standard k-fold is invalid for time series as it leaks future data into training. Prophet's built-in `cross_validation()` and `performance_metrics()` functions automate this, returning MAE, RMSE, and MAPE per forecast horizon.

When to use: Business data with clear seasonality, known holidays, and stakeholder-interpretable outputs.

4. AutoML for Time Series Forecasting

4.1 Why Machine Learning for Forecasting?

Traditional statistical models (ARIMA, Prophet) assume linear relationships and specific data structures. Machine learning overcomes these limitations:

- Captures non-linear interactions and complex feature relationships.
- Supports exogenous (external) variables natively — promotions, weather, economic indicators.
- Gradient Boosting models (LightGBM, XGBoost) consistently outperform ARIMA on many real-world datasets.
- Handles millions of rows efficiently; supports ensembling for better accuracy.
- Limitation: ML does not natively understand time — temporal structure must be injected through feature engineering.

4.2 Converting Time Series to Tabular Format

ML models expect a row-per-prediction table. The time series is transformed by creating features from its own structure:

- Lag Features: lag_1, lag_2, ..., lag_n — value n periods ago; captures autocorrelation.
- Rolling Statistics: Rolling mean (7d, 14d, 30d), rolling std (volatility), rolling min/max.
- Date/Time Features: Day of week, month, quarter, year, is_weekend, week of year.
- Exogenous Variables: Price, promotions, weather, holidays, competitor activity.

4.3 AutoML Pipeline

The automated pipeline eliminates manual model selection and tuning:

- Raw Data → Feature Engineering → Model Search → Hyperparameter Tuning → Ensemble & Stack → Best Model.

Hyperparameter search strategies:

- Grid Search: Exhaustive; covers all combinations but exponentially slow — $O(n^d)$.
- Random Search: 2–4× faster, similar accuracy; no learning from past trials.
- Bayesian Optimisation: Learns from prior trials; most efficient for expensive models.

4.4 AutoML Frameworks for Forecasting

- **Fast, lightweight; supports LightGBM, XGBoost, and time series.** FLAML (Microsoft):
- **Purpose-built for time series; 20+ models with multi-series support.** AutoTS:
- **Low-code; compare_models() for fast EDA + model selection.** PyCaret:
- **Ultra-fast AutoARIMA, AutoETS — 100× faster than statsmodels.** StatsForecast (Nixtla):

Walk-forward CV is mandatory: always split time series chronologically — never randomly. Use TimeSeriesSplit (sklearn) or Prophet's cross_validation().

4.5 Evaluation Metrics

Common metrics for assessing forecast accuracy:

- **Average absolute deviation; outliers weighted equally.** MAE (Mean Absolute Error):
- **Penalises large errors heavily; same units as target.** RMSE (Root Mean Squared Error):
- **Scale-free; useful for comparing across datasets. Undefined when actual = 0.** MAPE (Mean Absolute % Error):
- **Symmetric MAPE; avoids division-by-zero issues.** SMAPE:
- **MAE divided by naive baseline MAE; unitless and dataset-agnostic.** MASE:

5. AutoML + Feature Hashing for Multi-Series Forecasting

5.1 The Multi-Series Scalability Problem

Real-world forecasting often involves massive numbers of series — Amazon forecasts demand for 500K+ SKUs, energy grids manage 1M+ smart meter series. Training one model per series (local approach) fails at this scale:

- **Compute Explosion:** 500K ARIMA models = days of computation.
- **Cold Start:** New products/stores have no history — local models fail immediately.
- **No Cross-Learning:** Similar series (same category/region) share patterns, but local models cannot learn this shared structure.

5.2 Local vs Global Models

- **One model per series. Fully customised but does not scale beyond ~1,000 series. Cannot handle cold start.**Local Model:
- **One model trained across ALL series simultaneously. Scales to millions; learns shared seasonal/trend patterns; handles cold start via features. Requires rich feature representation of series identity.**Global Model:

5.3 The High-Cardinality Encoding Problem

To build a global model, each series must be identified to the model. Store/product IDs are high-cardinality categorical variables with potentially millions of unique values. Standard encoding methods fail:

- **One-Hot Encoding:** 1M categories → 1M columns — memory infeasible.
- **Label Encoding:** Implies false ordinal relationship (store_1 ≠ 'less than' store_2).

5.4 Feature Hashing (The Hashing Trick)

Feature hashing maps any string identifier to a fixed-size vector of K dimensions using a hash function:

```
Hash: Series ID → h(x) mod N → Fixed-Size Vector [0, ..., N] →  
ML Model Input
```

- **Output size K is fixed regardless of vocabulary size — no growing dictionary.**Memory Efficient:
- **New categories (new stores/products) are handled at inference without retraining the encoder.**No Pre-training:
- **Same input always maps to the same hash bucket — no randomness.**Deterministic:
- **Multiple IDs may map to the same bucket — usually a minor accuracy trade-off, controlled by choosing K (e.g., 128, 256, 512).**Collisions:

In Python, sklearn's `FeatureHasher(n_features=256, input_type='string')` implements this natively and integrates directly with any ML pipeline.

5.5 Full AutoML + Hashing Pipeline

- **Collect all series data in a single DataFrame (millions of rows).**Step 1:
- **Engineer lag, rolling, and date features per series.**Step 2:

- **Apply FeatureHasher to series identity columns (store_id, item_id, etc.) → K-dimensional hash vector.**Step 3:
- **Concatenate numeric features + hash vector.**Step 4:
- **AutoML search across LightGBM/XGBoost/CatBoost configurations.**Step 5:
- **One global model produces forecasts for all series simultaneously.**Step 6:

5.6 Cross-Series Learning

A global model trained on all series simultaneously learns shared structure implicitly. Series in the same category share seasonal patterns — the model learns these jointly. Practically, LightGBM trained on 10M rows across 100K series can outperform 100K individually tuned local models, while being 100–1000× faster to train.

When to use: 100K+ series, cold-start requirements, operational scale deployments, or when categories share seasonal/trend patterns.

6. Comparison of Methods

Criterion	Auto-ARIMA	Prophet	AutoML	AutoML+Hash
Scalability	Low	Low	Medium	High ✓
Interpretability	High ✓	High ✓	Medium	Low
Handles Seasonality	SARIMA	Built-in ✓✓	Via features	Via features
Handles Holidays	Manual	Built-in ✓	Via features	Via features
Multi-Series	1 per series	1 per series	1 per series	1 global ✓
Cold Start	✗	✗	Limited	Via hashing ✓
Exogenous Vars	Limited	Regressors	Native ✓✓	Native ✓✓
Best For	Univariate	Business+Events	Multi-feature	High-cardinality

7. Decision Guide & Key Takeaways

7.1 Choosing the Right Method

- 1–50 series, linear trend, classic patterns → Auto-ARIMA (SARIMA if seasonal).
- Business data with known holidays and strong seasonality → Prophet.
- Many exogenous features, non-linear relationships, moderate scale → AutoML (LightGBM/XGBoost).
- 100K+ series, cold start needed, operational scale → AutoML + Feature Hashing.
- When in doubt: baseline with naive seasonal model, then Auto-ARIMA, then LightGBM AutoML.
- Always validate using walk-forward cross-validation — never random splits.

7.2 Key Takeaways for Exam

- Automated forecasting removes the need for manual ACF/PACF reading and (p,d,q) selection.
 - Auto-ARIMA uses AIC/BIC to select the best ARIMA/SARIMA configuration algorithmically.
 - Prophet decomposes time series into trend, seasonality, holidays, and noise using a Bayesian additive model.
 - AutoML transforms time series into tabular data via feature engineering and then searches across ML models.
 - Feature Hashing encodes high-cardinality series IDs into a fixed-size vector, enabling one global ML model to forecast millions of series scalably.
 - Walk-forward CV is the only valid cross-validation strategy for time series — future data must never leak into training.
-